

Digital Nurture 5.0
Deep Skilling Handbook
Java FSE -Solution

WEEK - 1

Data Structures
And Algorithms

Exercise 1: Inventory Management System

Program:

```
import java.util.HashMap;
import java.util.Map;
class Product {
    private int productId;
    private String productName;
    private int quantity;
    private double price;
    public Product(int productId, String productName, int
quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }
    public int getProductId() {
        return productId;
    }
    public void setProductName(String productName) {
        this.productName = productName;
    }
    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
    public void setPrice(double price) {
        this.price = price;
    }
}
```

@Override

```
public String toString() {  
    return "Product ID: " + productId +  
        ", Name: " + productName +  
        ", Quantity: " + quantity +  
        ", Price: ₹" + price;  
}
```

```
}
```

```
public class InventoryManagementSystem {  
    private Map<Integer, Product> inventory;  
    public InventoryManagementSystem() {  
        inventory = new HashMap<>();  
    }  
    public void addProduct(Product product) {  
        inventory.put(product.getProductId(), product);  
        System.out.println("Product Added Successfully.");  
    }  
    public void updateProduct(int productId, String name, int  
quantity, double price) {  
        Product product = inventory.get(productId);  
        if (product != null) {  
            product.setProductName(name);  
            product.setQuantity(quantity);  
            product.setPrice(price);  
            System.out.println("Product Updated Successfully.");  
        } else {  
            System.out.println("Product Not Found.");  
        }  
    }  
    public void deleteProduct(int productId) {  
        if (inventory.remove(productId) != null) {  
            System.out.println("Product Deleted Successfully");  
        }  
    }  
}
```

```

    } else {
        System.out.println("Product Not Found.");
    }
}

public void displayProducts() {
    System.out.println("\nInventory Details:");
    for (Product product : inventory.values()) {
        System.out.println(product);
    }
}

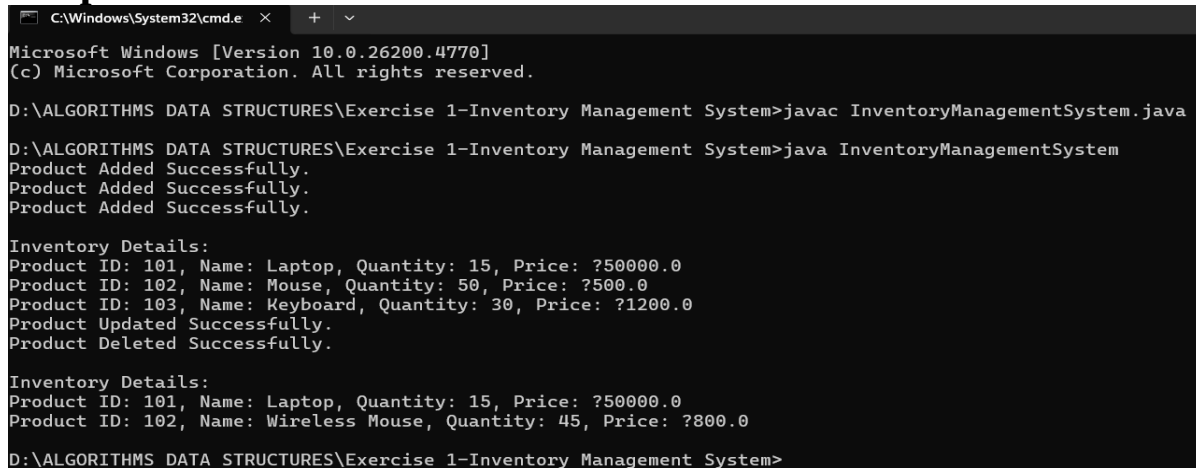
public static void main(String[] args) {
    InventoryManagementSystem ims = new
InventoryManagementSystem();

    ims.addProduct(new Product(101, "Laptop", 15, 50000));
    ims.addProduct(new Product(102, "Mouse", 50, 500));
    ims.addProduct(new Product(103, "Keyboard", 30, 1200));
    ims.displayProducts();

    ims.updateProduct(102, "Wireless Mouse", 45, 800);
    ims.deleteProduct(103);
    ims.displayProducts();
}
}

```

Output:



```

C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.4770]
(c) Microsoft Corporation. All rights reserved.

D:\ALGORITHMS DATA STRUCTURES\Exercise 1-Inventory Management System>javac InventoryManagementSystem.java

D:\ALGORITHMS DATA STRUCTURES\Exercise 1-Inventory Management System>java InventoryManagementSystem
Product Added Successfully.
Product Added Successfully.
Product Added Successfully.

Inventory Details:
Product ID: 101, Name: Laptop, Quantity: 15, Price: ?50000.0
Product ID: 102, Name: Mouse, Quantity: 50, Price: ?500.0
Product ID: 103, Name: Keyboard, Quantity: 30, Price: ?1200.0
Product Updated Successfully.
Product Deleted Successfully.

Inventory Details:
Product ID: 101, Name: Laptop, Quantity: 15, Price: ?50000.0
Product ID: 102, Name: Wireless Mouse, Quantity: 45, Price: ?800.0

D:\ALGORITHMS DATA STRUCTURES\Exercise 1-Inventory Management System>

```

Exercise 2: E-commerce Platform Search Function

PROGRAM:

```
import java.util.Arrays;

import java.util.Comparator;

class Product {

    int productId;

    String productName;

    String category;

    public Product(int productId, String productName, String category) {

        this.productId = productId;

        this.productName = productName;

        this.category = category;

    }

    @Override

    public String toString() {

        return "Product ID: " + productId +

            ", Name: " + productName +

            ", Category: " + category;

    }

}

public class ECommerceSearch {

    public static Product linearSearch(Product[] products, int targetId) {

        for (Product product : products) {
```

```
        if (product.productId == targetId) {

            return product;

        }

    }

    return null;

}

public static Product binarySearch(Product[] products, int targetId) {

    int left = 0;

    int right = products.length - 1;

    while (left <= right) {

        int mid = left + (right - left) / 2;

        if (products[mid].productId == targetId) {

            return products[mid];

        }

        if (products[mid].productId < targetId) {

            left = mid + 1;

        } else {

            right = mid - 1;

        }

    }

    return null;

}

public static void main(String[] args) {

    Product[] products = {
```

```
        new Product(105, "Laptop", "Electronics"),
        new Product(101, "Mobile", "Electronics"),
        new Product(103, "Shoes", "Fashion"),
        new Product(102, "Watch", "Accessories"),
        new Product(104, "Bag", "Fashion")
    };

    int searchId = 103;

    Product linearResult = linearSearch(products, searchId);

    System.out.println("Linear Search Result:");

    if (linearResult != null) {

        System.out.println(linearResult);

    } else {

        System.out.println("Product Not Found");

    }

    Arrays.sort(products, Comparator.comparingInt(p -> p.productId));

    Product binaryResult = binarySearch(products, searchId);

    System.out.println("\nBinary Search Result:");

    if (binaryResult != null) {

        System.out.println(binaryResult);

    } else {

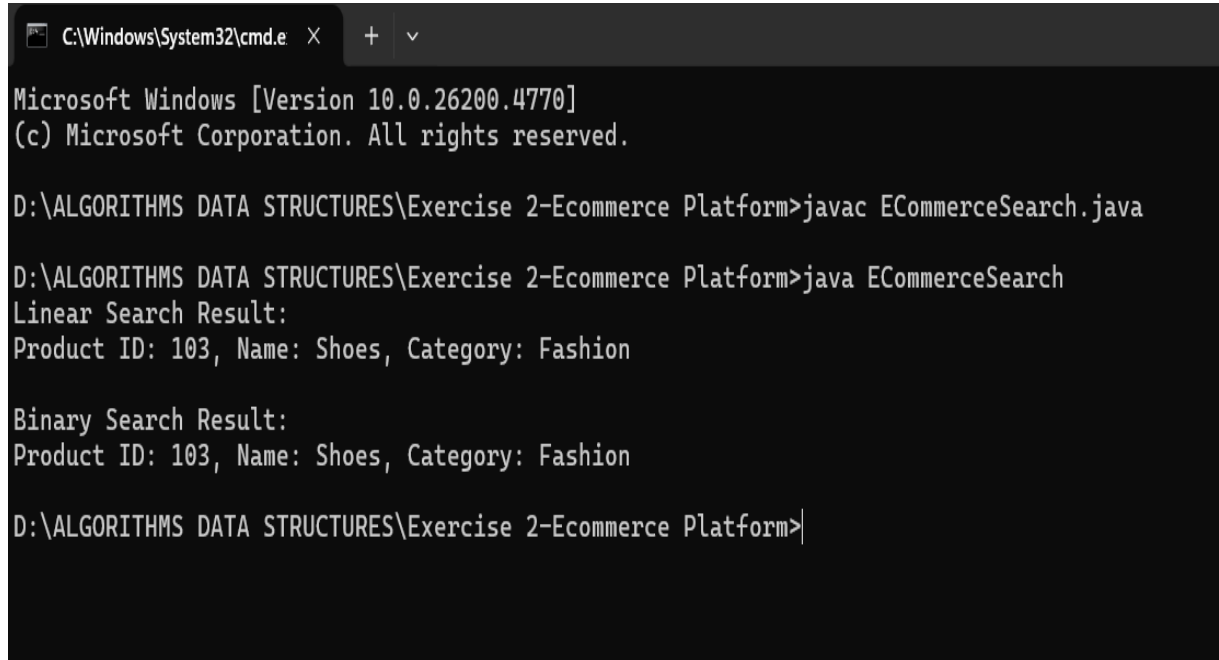
        System.out.println("Product Not Found");

    }

}

}
```

OUTPUT:



```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.4770]
(c) Microsoft Corporation. All rights reserved.

D:\ALGORITHMS DATA STRUCTURES\Exercise 2-Ecommerce Platform>javac ECommerceSearch.java

D:\ALGORITHMS DATA STRUCTURES\Exercise 2-Ecommerce Platform>java ECommerceSearch
Linear Search Result:
Product ID: 103, Name: Shoes, Category: Fashion

Binary Search Result:
Product ID: 103, Name: Shoes, Category: Fashion

D:\ALGORITHMS DATA STRUCTURES\Exercise 2-Ecommerce Platform>
```

Exercise 3: Sorting Customer Orders

PROGRAM:

```
class Order {
    int orderId;
    String customerName;
    double totalPrice;
    public Order(int orderId, String customerName, double totalPrice) {
        this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }
    @Override
    public String toString() {
        return "Order ID: " + orderId +
            ", Customer: " + customerName +
            ", Total Price: ₹" + totalPrice;
    }
}

public class OrderSortingSystem {
    public static void bubbleSort(Order[] orders) {
        int n = orders.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {

                if (orders[j].totalPrice > orders[j + 1].totalPrice) {
                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                }
            }
        }
    }

    public static void quickSort(Order[] orders, int low, int high) {
        if (low < high) {
```

```

int pivotIndex = partition(orders, low, high);
    quickSort(orders, low, pivotIndex - 1);
    quickSort(orders, pivotIndex + 1, high);
}
}

public static int partition(Order[] orders, int low, int high) {
    double pivot = orders[high].totalPrice;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (orders[j].totalPrice <= pivot) {
            i++;
            Order temp = orders[i];
            orders[i] = orders[j];
            orders[j] = temp;
        }
    }
    Order temp = orders[i + 1];
    orders[i + 1] = orders[high];
    orders[high] = temp;
    return i + 1;
}

public static void displayOrders(Order[] orders) {
    for (Order order : orders) {
        System.out.println(order);
    }
}

public static void main(String[] args) {
    Order[] orders1 = {
        new Order(101, "Janani", 2500),
        new Order(102, "Shifa", 1200),
        new Order(103, "Geko", 4500),
        new Order(104, "Bommi", 1800),
        new Order(105, "Anusha", 3200)
    };
    Order[] orders2 = {
        new Order(101, "Janani", 2500),

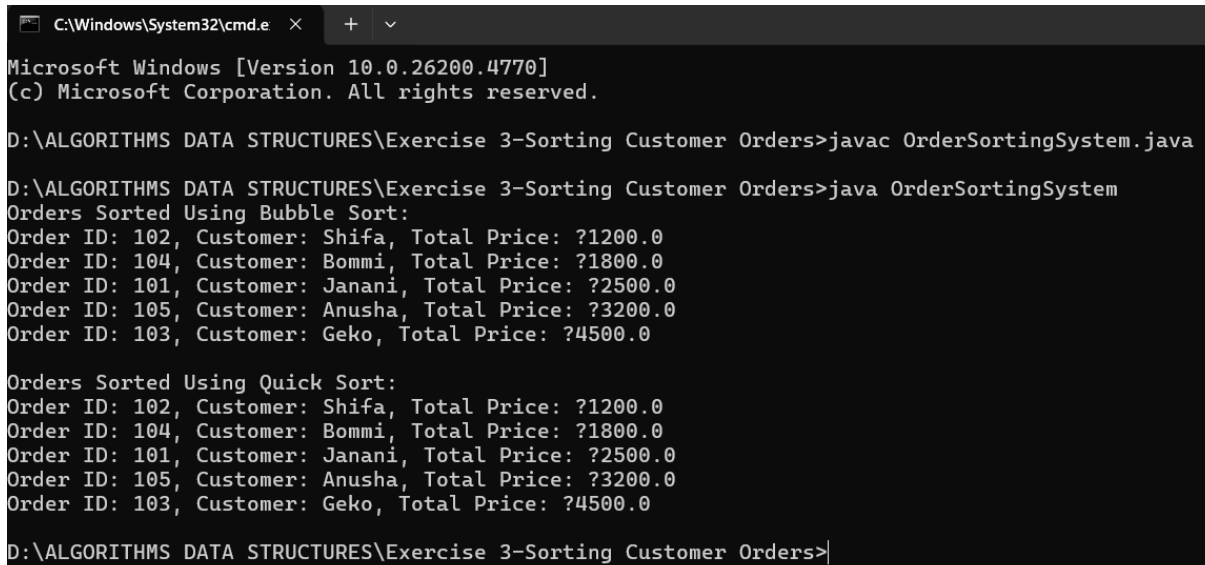
```

```

new Order(102, "Shifa", 1200),
    new Order(103, "Geko", 4500),
    new Order(104, "Bommi", 1800),
    new Order(105, "Anusha", 3200)
};
System.out.println("Orders Sorted Using Bubble Sort:");
bubbleSort(orders1);
displayOrders(orders1);
System.out.println("\nOrders Sorted Using Quick Sort:");
quickSort(orders2, 0, orders2.length - 1);
displayOrders(orders2);
}
}

```

OUTPUT:



```

C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.4770]
(c) Microsoft Corporation. All rights reserved.

D:\ALGORITHMS DATA STRUCTURES\Exercise 3-Sorting Customer Orders>javac OrderSortingSystem.java

D:\ALGORITHMS DATA STRUCTURES\Exercise 3-Sorting Customer Orders>java OrderSortingSystem
Orders Sorted Using Bubble Sort:
Order ID: 102, Customer: Shifa, Total Price: ?1200.0
Order ID: 104, Customer: Bommi, Total Price: ?1800.0
Order ID: 101, Customer: Janani, Total Price: ?2500.0
Order ID: 105, Customer: Anusha, Total Price: ?3200.0
Order ID: 103, Customer: Geko, Total Price: ?4500.0

Orders Sorted Using Quick Sort:
Order ID: 102, Customer: Shifa, Total Price: ?1200.0
Order ID: 104, Customer: Bommi, Total Price: ?1800.0
Order ID: 101, Customer: Janani, Total Price: ?2500.0
Order ID: 105, Customer: Anusha, Total Price: ?3200.0
Order ID: 103, Customer: Geko, Total Price: ?4500.0

D:\ALGORITHMS DATA STRUCTURES\Exercise 3-Sorting Customer Orders>

```

Exercise 4: Employee Management System

PROGRAM:

```
class Employee {
    int employeeId;
    String name;
    String position;
    double salary;
    public Employee(int employeeId, String name, String position, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }
    @Override
    public String toString() {
        return "Employee ID: " + employeeId +
            ", Name: " + name +
            ", Position: " + position +
            ", Salary: ₹" + salary;
    }
}

public class EmployeeManagementSystem {
    private Employee[] employees;
    private int count;
    public EmployeeManagementSystem(int size) {
        employees = new Employee[size];
        count = 0;
    }
    public void addEmployee(Employee employee) {
        if (count < employees.length) {
            employees[count] = employee;
            count++;
            System.out.println("Employee Added Successfully.");
        } else {
```

```
        System.out.println("Employee Array is Full.");
    }
}

public Employee searchEmployee(int employeeId) {
    for (int i = 0; i < count; i++) {
        if (employees[i].employeeId == employeeId) {
            return employees[i];
        }
    }
    return null;
}

public void displayEmployees() {
    System.out.println("\nEmployee Records:");
    for (int i = 0; i < count; i++) {
        System.out.println(employees[i]);
    }
}

public void deleteEmployee(int employeeId) {
    int index = -1;
    for (int i = 0; i < count; i++) {
        if (employees[i].employeeId == employeeId) {
            index = i;
            break;
        }
    }
    if (index == -1) {
        System.out.println("Employee Not Found.");
        return;
    }
    for (int i = index; i < count - 1; i++) {
        employees[i] = employees[i + 1];
    }
    employees[count - 1] = null;
    count--;
    System.out.println("Employee Deleted Successfully.");
}
```

```
public static void main(String[] args) {  
    EmployeeManagementSystem system =  
        new EmployeeManagementSystem(10);  
    system.addEmployee(  
        new Employee(101, "Janani", "Developer", 50000));  
    system.addEmployee(  
        new Employee(102, "Rahul", "Tester", 40000));  
    system.addEmployee(  
        new Employee(103, "Priya", "Manager", 70000));  
    system.displayEmployees();  
    System.out.println("\nSearching Employee ID 102:");  
    Employee emp = system.searchEmployee(102);  
    if (emp != null) {  
        System.out.println(emp);  
    } else {  
        System.out.println("Employee Not Found.");  
    }  
    system.deleteEmployee(102);  
    system.displayEmployees();  
}  
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.4770]
(c) Microsoft Corporation. All rights reserved.

D:\ALGORITHMS DATA STRUCTURES\Exercise 4- Employee Management System>javac EmployeeManagementSystem.java

D:\ALGORITHMS DATA STRUCTURES\Exercise 4- Employee Management System>java EmployeeManagementSystem
Employee Added Successfully.
Employee Added Successfully.
Employee Added Successfully.

Employee Records:
Employee ID: 101, Name: Janani, Position: Developer, Salary: ?50000.0
Employee ID: 102, Name: Rahul, Position: Tester, Salary: ?40000.0
Employee ID: 103, Name: Priya, Position: Manager, Salary: ?70000.0

Searching Employee ID 102:
Employee ID: 102, Name: Rahul, Position: Tester, Salary: ?40000.0
Employee Deleted Successfully.

Employee Records:
Employee ID: 101, Name: Janani, Position: Developer, Salary: ?50000.0
Employee ID: 103, Name: Priya, Position: Manager, Salary: ?70000.0

D:\ALGORITHMS DATA STRUCTURES\Exercise 4- Employee Management System>
```

Exercise 5: Task Management System

PROGRAM:

```
class Task {
    int taskId;
    String taskName;
    String status;
    public Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
    }
    @Override
    public String toString() {
        return "Task ID: " + taskId +
            ", Task Name: " + taskName +
            ", Status: " + status;
    }
}

class Node {
    Task task;
    Node next;
    public Node(Task task) {
        this.task = task;
        this.next = null;
    }
}

public class TaskManagementSystem {
    private Node head;
    public void addTask(Task task) {
        Node newNode = new Node(task);
        if (head == null) {
            head = newNode;
        } else {
            Node current = head;
```

```
while (current.next != null) {
    current = current.next;
}
current.next = newNode;
}
System.out.println("Task Added Successfully.");
}

public Task searchTask(int taskId) {
    Node current = head;
    while (current != null) {
        if (current.task.taskId == taskId) {
            return current.task;
        }
        current = current.next;
    }
    return null;
}

public void displayTasks() {
    System.out.println("\nTask List:");
    Node current = head;
    while (current != null) {
        System.out.println(current.task);
        current = current.next;
    }
}

public void deleteTask(int taskId) {
    if (head == null) {
        System.out.println("Task List is Empty.");
        return;
    }
    if (head.task.taskId == taskId) {
        head = head.next;
        System.out.println("Task Deleted Successfully.");
        return;
    }
    Node current = head;
```

```
        while (current.next != null &&
            current.next.task.taskId != taskId) {
            current = current.next;
        }
        if (current.next == null) {
            System.out.println("Task Not Found.");
        } else {
            current.next = current.next.next;
            System.out.println("Task Deleted Successfully.");
        }
    }
}

public static void main(String[] args) {
    TaskManagementSystem system = new TaskManagementSystem();
    system.addTask(new Task(101, "Design UI", "Pending"));
    system.addTask(new Task(102, "Develop Backend", "In Progress"));
    system.addTask(new Task(103, "Testing", "Pending"));
    system.displayTasks();
    System.out.println("\nSearching Task ID 102:");
    Task task = system.searchTask(102);
    if (task != null) {
        System.out.println(task);
    } else {
        System.out.println("Task Not Found.");
    }
    system.deleteTask(102);
    system.displayTasks();
}
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.4770]
(c) Microsoft Corporation. All rights reserved.

D:\ALGORITHMS DATA STRUCTURES\Exercise 5 -Task Management System>javac TaskManagementSystem.java

D:\ALGORITHMS DATA STRUCTURES\Exercise 5 -Task Management System>java TaskManagementSystem
Task Added Successfully.
Task Added Successfully.
Task Added Successfully.

Task List:
Task ID: 101, Task Name: Design UI, Status: Pending
Task ID: 102, Task Name: Develop Backend, Status: In Progress
Task ID: 103, Task Name: Testing, Status: Pending

Searching Task ID 102:
Task ID: 102, Task Name: Develop Backend, Status: In Progress
Task Deleted Successfully.

Task List:
Task ID: 101, Task Name: Design UI, Status: Pending
Task ID: 103, Task Name: Testing, Status: Pending

D:\ALGORITHMS DATA STRUCTURES\Exercise 5 -Task Management System>
```

Exercise 6: Library Management System

PROGRAM:

```
import java.util.Arrays;

import java.util.Comparator;

class Book {

    int bookId;

    String title;

    String author;

    public Book(int bookId, String title, String author) {

        this.bookId = bookId;

        this.title = title;

        this.author = author;

    }

    @Override

    public String toString() {

        return "Book ID: " + bookId +

            ", Title: " + title +

            ", Author: " + author;

    }

}

public class LibraryManagementSystem {

    public static Book linearSearch(Book[] books, String targetTitle) {

        for (Book book : books) {
```

```
        if (book.title.equalsIgnoreCase(targetTitle)) {

            return book;

        }

    }

    return null;

}

public static Book binarySearch(Book[] books, String targetTitle) {

    int left = 0;

    int right = books.length - 1;

    while (left <= right) {

        int mid = (left + right) / 2;

        int comparison =

            books[mid].title.compareToIgnoreCase(targetTitle);

        if (comparison == 0) {

            return books[mid];

        }

        if (comparison < 0) {

            left = mid + 1;

        } else {

            right = mid - 1;

        }

    }

    return null;

}

public static void main(String[] args) {

    Book[] books = {
```

```
        new Book(101, "Java Programming", "James Gosling"),
        new Book(102, "Data Structures", "Mark Allen"),
        new Book(103, "Operating Systems", "Andrew Tanenbaum"),
        new Book(104, "Computer Networks", "Behrouz Forouzan"),
        new Book(105, "Database Systems", "Elmasri")
    };

    String searchTitle = "Data Structures";

    Book linearResult = linearSearch(books, searchTitle);

    System.out.println("Linear Search Result:");

    if (linearResult != null) {

        System.out.println(linearResult);
    } else {

        System.out.println("Book Not Found");
    }

    Arrays.sort(books,

        Comparator.comparing(book -> book.title.toLowerCase()));

    Book binaryResult = binarySearch(books, searchTitle);

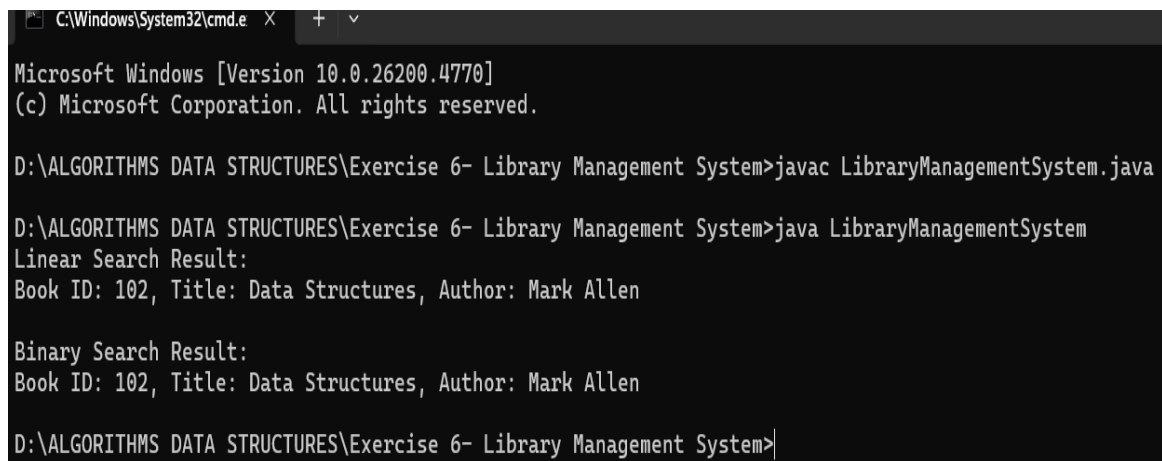
    System.out.println("\nBinary Search Result:");

    if (binaryResult != null) {

        System.out.println(binaryResult);
    } else {

        System.out.println("Book Not Found");
    }
}
}
```

OUTPUT:



```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.4770]
(c) Microsoft Corporation. All rights reserved.

D:\ALGORITHMS DATA STRUCTURES\Exercise 6- Library Management System>javac LibraryManagementSystem.java

D:\ALGORITHMS DATA STRUCTURES\Exercise 6- Library Management System>java LibraryManagementSystem
Linear Search Result:
Book ID: 102, Title: Data Structures, Author: Mark Allen

Binary Search Result:
Book ID: 102, Title: Data Structures, Author: Mark Allen

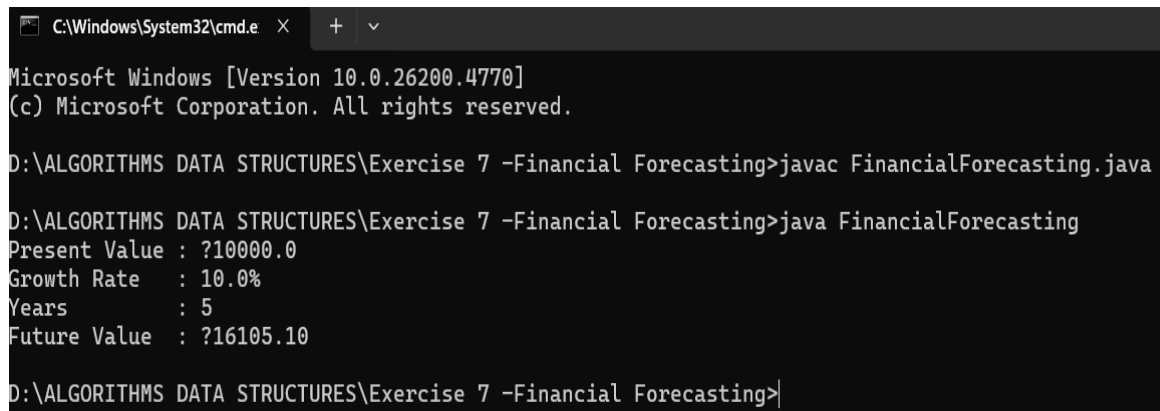
D:\ALGORITHMS DATA STRUCTURES\Exercise 6- Library Management System>
```

Exercise 7: Financial Forecasting

PROGRAM:

```
public class FinancialForecasting {  
    public static double calculateFutureValue(double presentValue,double growthRate,int  
years) {  
  
        if (years == 0) {  
            return presentValue;  
        }  
        return calculateFutureValue(  
            presentValue,  
            growthRate,  
            years - 1) * (1 + growthRate);  
    }  
    public static void main(String[] args) {  
        double presentValue = 10000;  
        double growthRate = 0.10;  
        int years = 5;  
        double futureValue =  
            calculateFutureValue(presentValue, growthRate, years);  
  
        System.out.println("Present Value : ₹" + presentValue);  
        System.out.println("Growth Rate : " + (growthRate * 100) + "%");  
        System.out.println("Years : " + years);  
        System.out.printf("Future Value : ₹%.2f\n", futureValue);  
    }  
}
```

OUTPUT:



```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.4770]
(c) Microsoft Corporation. All rights reserved.

D:\ALGORITHMS DATA STRUCTURES\Exercise 7 -Financial Forecasting>javac FinancialForecasting.java

D:\ALGORITHMS DATA STRUCTURES\Exercise 7 -Financial Forecasting>java FinancialForecasting
Present Value : ?10000.0
Growth Rate   : 10.0%
Years         : 5
Future Value  : ?16105.10

D:\ALGORITHMS DATA STRUCTURES\Exercise 7 -Financial Forecasting>|
```